

CHEF: A Hybrid Recipe for Item-Level Receipt Parsing

OCR-Free vs. OCR-Dependent Architectures for Automated Bill Splitting

Ishneet Chadha, Manikandan Gunaseelan

University of Colorado Boulder

CSCI 5922 — Neural Networks and Deep Learning

Abstract

Automated bill splitting requires a receipt parser that extracts individual line items and prices, not just document-level fields such as date and total. This report studies two competing architectures for that problem: Donut, an OCR-free image-to-JSON transformer, and LayoutLMv3, an OCR-dependent multimodal token classifier. We evaluate both models on the CORD test split using entity-level F1, line-item matching, and a receipt checksum metric, then introduce CHEF, a simple field-routed hybrid that uses LayoutLMv3 for line-item fields and Donut for aggregate fields. With human-verified CORD OCR, LayoutLMv3 substantially outperforms Donut on item names, item prices, and counts, while Donut is slightly better on subtotals and totals. CHEF combines these complementary strengths, achieving the best normalized micro-F1 of 0.940. However, replacing gold OCR with Tesseract drops LayoutLMv3 from 0.937 to 0.314 micro-F1, showing that much of the apparent advantage of OCR-dependent receipt parsing comes from high-quality OCR rather than the downstream model alone.

that the resulting shares are consistent with the receipt total.

This workflow depends on a finer-grained version of receipt understanding than the one emphasized by many receipt benchmarks. A model that extracts the vendor name, purchase date, and total can be useful for accounting, but it is insufficient for item-level bill splitting. The system must correctly read each item name, read its price, and associate the correct price with the correct item. A mistake in that association is not a minor formatting error: it changes who pays what. Receipt parsing is also difficult in practice because receipts contain small fonts, low contrast thermal printing, abbreviations, tax and service-charge lines, irregular spacing, and diverse point-of-sale templates.

Modern document understanding systems follow two broad paradigms. OCR-dependent systems first use an OCR engine to detect text and bounding boxes, then pass the recognized tokens and spatial coordinates to a downstream model. LayoutLM and its successors are representative of this approach [3, 4, 2]. The strength of the paradigm is that it exposes explicit text and layout to the neural model; the weakness is that errors in OCR become hard limits on downstream extraction. OCR-free systems instead treat the document image as the primary input and directly generate structured output. Donut is the representative model in this report: it encodes pixels with a vision transformer and decodes a JSON-like answer sequence without an external OCR step [1]. This avoids explicit OCR error propagation, but may introduce other failures such as hallucinated strings or malformed output.

Our original project question was whether OCR-free or OCR-dependent architectures are better suited for item-level receipt parsing. During the project, the results showed a more nuanced answer. LayoutLMv3 is much stronger than Donut on item-level fields when it receives CORD’s human-

Keywords: receipt parsing, document understanding, OCR, Donut, LayoutLMv3, information extraction, bill splitting, neural networks

1 Introduction

Splitting shared expenses is a common task in restaurants, travel, and shared households. Existing payment and debt-tracking applications make it easy to record the final transfer, but they still usually require users to manually enter itemized amounts when people did not split the bill equally. This manual step is tedious, easy to get wrong, and especially frustrating on long receipts. A practical automated bill-splitting system would let a user photograph a receipt, recover the purchased items and prices, assign payment to those items, and check

verified OCR annotations, but Donut is slightly stronger on aggregate fields such as subtotal and total. This motivated CHEF (Composed Hybrid for Extraction and Field-routing), a simple hybrid system that routes line-item fields to LayoutLMv3 and aggregate fields to Donut. The hybrid does not retrain either model; it only composes their outputs by field.

This report makes three contributions. First, it evaluates OCR-free and OCR-dependent receipt parsing specifically at the item level, using metrics that reflect bill-splitting needs rather than only document-level extraction. Second, it shows that the two paradigms have complementary strengths: token classification is better for dense menu fields, while schema-constrained generation is competitive for aggregate totals. Third, it includes an OCR-source ablation showing that standard OCR-dependent results with gold OCR can substantially overstate deployable performance. With Tesseract in place of CORD’s verified OCR, LayoutLMv3’s normalized micro-F1 falls by roughly 0.62 absolute F1.

2 Related Work

Receipt understanding benchmarks provide the data foundation for this work. The ICDAR 2019 SROIE competition introduced a scanned receipt benchmark covering text localization, OCR, and key information extraction for fields such as company, date, address, and total [6]. CORD was designed for post-OCR receipt parsing and provides fine-grained semantic annotations for receipt fields including menu names, counts, prices, subtotals, and totals [5]. WildReceipt later emphasized generalization to receipts with varied templates and categories [7]. These datasets made receipt extraction measurable, but the most commonly reported results are often aggregate field scores. Our work uses CORD to focus on the specific item-level fields required by bill splitting and evaluates whether item names and prices are recovered together.

OCR-dependent document models combine recognized text, 2D layout, and sometimes visual features. LayoutLM introduced joint pre-training over text and layout for document image understanding [3]. LayoutLMv2 added stronger multimodal pre-training with image features and spatially aware objectives [4]. LayoutLMv3 unified

text and image masking objectives and has become a strong baseline for visually rich document understanding [2]. Graph-based approaches such as Spatial Dual-Modality Graph Reasoning model relationships among OCR tokens using both semantic and spatial edges [7]. These systems are powerful when OCR is accurate, but their results conflate the quality of the OCR source with the quality of the downstream model. Our OCR-source ablation isolates this issue by comparing LayoutLMv3 with CORD’s verified word boxes against the same model driven by Tesseract OCR.

OCR-free document understanding has emerged as an alternative to explicit OCR pipelines. Donut proposed an OCR-free document understanding transformer that maps document pixels directly to a structured target sequence [1]. It uses a Swin Transformer visual encoder [8] and a text decoder to generate task-specific outputs. Pix2Struct extended the broader image-to-structure idea by pre-training on screenshot parsing tasks [9]. OCR-free systems are attractive for receipts because they avoid brittle intermediate OCR decisions, but their autoregressive generation can produce plausible text that is not actually present in the image. Our results confirm both sides: Donut is robust to the absence of OCR and performs well on totals, but it underperforms LayoutLMv3 on dense item fields and occasionally produces severe numeric hallucinations.

Receipt scanning has also been studied from an application perspective. Open-source and commercial bill-splitting tools such as OCRcpt and Receipt Hacker use OCR APIs to populate receipt items for user-facing split interfaces [10, 11]. These systems demonstrate the usefulness of receipt scanning but usually treat OCR as a black-box service and do not provide controlled comparisons of architectural choices. Our work is closer to an empirical model study than a finished mobile application: we evaluate the extraction component that such an application would rely on and identify where a practical system should expect failures.

3 Methods

3.1 Task Definition and Data

We formulate item-level receipt parsing as structured information extraction from a receipt image. For each receipt, the desired output is a set of menu fields and aggregate fields.

The menu fields are `menu.nm`, `menu.price`,
and `menu.cnt`; these represent item names,
item prices, and quantities. The aggregate
fields are `sub_total.subtotal_price` and
`total.total_price`; these provide receipt-
level consistency checks. For the bill-splitting use
case, the central output is a list of item records
containing a name, price, and optional count.

The primary dataset is CORD, using its
800/100/100 train, validation, and test split.
CORD includes receipt images, word-level text
annotations, bounding boxes, and semantic labels.
We evaluate on the 100-receipt test split. We nor-
malize predictions for the main reported scores
by lowercasing text and removing whitespace, be-
cause minor spacing and casing differences do not
affect bill splitting. We also preserve exact-match
metrics as a stricter diagnostic.

3.2 Donut: OCR-Free Parsing

Donut is our OCR-free model. It processes the
receipt image directly and generates a JSON-
like structured sequence. We use the public
`donut-base-finetuned-cord-v2` check-
point, which is already fine-tuned for CORD-style
receipt outputs. Donut’s advantage for this study
is that it avoids any dependence on OCR bounding
boxes. If a receipt has distorted text or an OCR
engine fails to localize words, Donut can still use
visual context to generate a structured answer.

At inference time, we decode Donut’s output
into field lists for the five target fields. We then
evaluate the decoded strings against the CORD
ground truth using the same matching code used
for LayoutLMv3. This keeps the comparison cen-
tered on final extracted values rather than model-
specific output formats. Since Donut generates a
structured sequence, it can naturally produce subto-
tal and total values even when they are visually
separated from menu items.

The main practical complication for Donut is
output validation. The decoder can generate a
sequence that is syntactically close to the target
schema but still contains missing fields, repeated
fields, or values whose numeric scale is wrong.
We therefore treat Donut as a structured predic-
tor rather than as a guaranteed parser: generated
fields are normalized, empty strings are discarded,
and numeric fields are parsed only when the ex-
tracted text can be converted into a receipt-like

amount. This validation does not correct Donut’s
predictions, but it prevents malformed output from
breaking the shared evaluation pipeline.

3.3 LayoutLMv3: OCR-Dependent Token Classification

LayoutLMv3 is our OCR-dependent model. It
receives token text, token bounding boxes,
and image features, then predicts a seman-
tic label for each token. We fine-tuned
`microsoft/layoutlmv3-base` for BIO to-
ken classification over the selected CORD fields.
Training was performed on a MacBook Air using
MPS acceleration with the visual backbone frozen
and eight training epochs. The frozen visual back-
bone was a pragmatic compute constraint: it kept
training feasible while still letting the transformer
learn text-layout associations for receipt fields.

The label-construction step is important for
CORD. Some tokens are keys, such as “TOTAL”,
while nearby tokens are values, such as the numeric
total. In an early version, key tokens and value to-
kens were labeled identically for aggregate fields.
This caused the model to emit strings such as “TO-
TAL 60.000” when the ground truth value was
only “60.000”. We therefore filtered `is_key=1`
tokens to the background class for value extraction.
This change dramatically improved aggregate field
scores and is reported as an ablation in Section 4.3.

We evaluate LayoutLMv3 under two OCR con-
ditions. The first uses CORD’s human-verified
word boxes and transcripts. This is the standard
setting for post-OCR parsing on CORD, but it
represents an optimistic upper bound because a de-
ployed application will not have those annotations.
The second condition replaces the input text and
boxes with Tesseract v5 OCR on the same images.
The downstream model and evaluation protocol are
kept fixed, so the performance difference estimates
the practical cost of OCR quality.

For token classification, each word is assigned
a BIO label for one of the target value fields or
the background class. Bounding boxes are nor-
malized to the coordinate convention expected by
LayoutLMv3. At prediction time, adjacent tokens
with compatible BIO labels are joined into field
strings in reading order. This reconstruction step is
deliberately simple because the goal is to compare
model behavior, not to hide errors behind a com-
plex post-processor. In particular, if OCR splits an

item into unusual fragments or places a price on the wrong line, the post-processing step does not try to infer the intended value from a receipt-specific grammar.

3.4 CHEF: Field-Routed Hybrid Composition

CHEF is a lightweight hybrid built after observing that the two models win on different fields. It uses LayoutLMv3 for `menu.nm`, `menu.price`, and `menu.cnt`, and Donut for `sub_total.subtotal_price` and `total.total_price`. The intuition is that item-level extraction is a dense local token-labeling problem, while aggregate extraction is a more global schema-constrained generation problem. CHEF does not train a new neural network; it composes the field outputs from the two existing models.

We include a reversed-routing control to test whether the gain comes from meaningful field routing or merely from combining two systems. The reversed system uses Donut for menu fields and LayoutLMv3 for aggregate fields. If arbitrary ensembling were responsible for the improvement, this reversed system should also improve. Instead, it performs worse than CHEF and worse than LayoutLMv3 alone, supporting the interpretation that the routing direction is responsible for the gain.

The hybrid design also reflects how a user-facing bill-splitting tool would consume the parse. Menu fields determine the editable list of items that users assign to people. Aggregate fields act more like constraints: the extracted subtotal and total can be compared against the sum of the menu prices and used to warn about missing tax, tips, discounts, or extraction mistakes. Because these two uses are different, there is no requirement that a single model provide every field. A composition that improves the reliability of each field group is acceptable as long as the output remains auditable.

3.5 Evaluation Metrics

We report entity-level precision, recall, and F1 for each field. A prediction is counted as correct when its normalized string matches a ground-truth string for the same field. Micro-F1 aggregates true positives, predictions, and gold values across all five fields. This metric gives a compact summary of extraction quality while still allowing field-level diagnosis.

We also report line-item matching accuracy. A ground-truth line item is counted as recovered only when both its item name and item price are predicted and associated correctly. This is stricter than separate entity F1 because a correct item name paired with the wrong price is not useful for bill splitting. In our implementation, line-item matching uses the extracted menu field structure and normalized strings.

Finally, we compute checksum consistency. For receipts where item prices and totals are parseable as numbers, we compare the sum of extracted item prices against the predicted total and count the receipt as consistent when the relative difference is within 5%. This metric is not a pure accuracy measure because real receipts include tax, discounts, tips, and service fees, so even ground truth does not always satisfy item-sum equals total. It is still useful as a deployment diagnostic: very large checksum errors reveal numeric hallucinations or missing fields that field-level F1 may hide.

All reported metrics are computed on the same cached prediction files for each model. This made the ablations reproducible: after predictions were generated, changes to normalization, routing, or scoring could be rerun without changing the model outputs. That separation was useful because several early differences between systems came from evaluation details rather than architecture. For example, exact string matching penalized harmless whitespace differences, while normalized matching better reflected the bill-splitting use case. We therefore report normalized scores as the headline numbers and use exact scores as supporting diagnostics.

4 Experiments

4.1 Main CORD Results

Table 1 shows normalized field-level F1 on the CORD test split. LayoutLMv3 with gold OCR is strongest on all three menu fields: item names, prices, and counts. The largest gap is on item names, where LayoutLMv3 reaches 0.899 F1 and Donut reaches 0.791 F1. This reflects the advantage of token-level classification for dense receipt lines. When the model has reliable word boxes and transcripts, assigning semantic tags to nearby tokens is easier than generating every item string from pixels.

Table 1: Normalized F1 on the CORD test split ($n = 100$). CHEF routes menu fields to LayoutLMv3 and aggregate fields to Donut.

Field	LLM	Donut	CHEF
menu.nm	0.899	0.791	0.899
menu.price	0.968	0.866	0.968
menu.cnt	0.980	0.968	0.980
subtotal_price	0.882	0.901	0.901
total_price	0.898	0.917	0.917
Micro-F1	0.937	0.878	0.940
Line-item match	0.886	0.732	0.886

Table 2: OCR-source ablation on the CORD test split. Replacing verified CORD OCR with Tesseract causes a large drop for the OCR-dependent model.

System	Micro	Line
LayoutLMv3 + gold OCR	0.937	0.886
LayoutLMv3 + Tesseract	0.314	0.061
Donut, no OCR	0.878	0.732

ble but not a normalized match. On prices, LayoutLMv3 is especially strong because prices are short local tokens with distinctive layout positions. Donut’s price score is lower, and its errors matter more than they might appear because a wrong digit changes the downstream split.

Counts are the easiest menu field for both systems. Many receipts either omit counts or use simple integer quantities, so the model has fewer string variants to recover. Subtotals and totals are different: they are often introduced by explicit keys, appear near the bottom of the receipt, and have strong schema regularity. This explains why Donut can win on aggregate fields despite losing on line items. Its decoder can exploit global document regularities, while LayoutLMv3 must separate value tokens from nearby key tokens, tax labels, and other bottom-of-receipt amounts.

4.2 OCR-Source Ablation

The main LayoutLMv3 result uses CORD’s human-verified OCR, which is useful for comparing post-OCR parsers but optimistic for deployment. To measure this dependency, we replaced gold OCR with Tesseract v5 while keeping the same LayoutLMv3 architecture, images, and evaluation code. Table 2 reports the result.

The drop is severe: normalized micro-F1 falls from 0.937 to 0.314, and line-item matching falls from 0.886 to 0.061. This means the impressive gold-OCR LayoutLMv3 result is not solely a property of the neural classifier. It also depends on the availability of high-quality OCR transcripts and boxes. Under realistic OCR, Donut becomes substantially stronger despite being worse than gold-OCR LayoutLMv3 in the main table.

This ablation changes how we interpret the architecture comparison. If a deployed application can use an excellent OCR service, a LayoutLM-style parser may be preferable for menu fields. If the application relies on a weaker local OCR engine, the OCR-free model may be more robust.

Donut is slightly stronger on aggregate fields. It obtains 0.901 F1 on subtotal and 0.917 F1 on total, compared with 0.882 and 0.898 for LayoutLMv3. These fields are sparse, semantically prominent, and highly constrained by receipt schemas. Donut’s sequence decoder appears to benefit from that schema regularity. The result is not that one paradigm dominates the other, but that the best architecture depends on the field type.

CHEF achieves the best normalized micro-F1, 0.940. The absolute gain over LayoutLMv3 alone is small, but it is consistent with the per-field trend: CHEF keeps LayoutLMv3’s strong menu extraction while replacing the weaker aggregate fields with Donut’s stronger predictions. The line-item matching score remains equal to LayoutLMv3 because CHEF’s line items come from LayoutLMv3. This is desirable for the target application: the hybrid improves receipt-level fields without sacrificing item associations.

The exact-match scores follow the same trend, though text normalization helps both models. Donut benefits more from normalization on menu names and counts, suggesting that some of its errors are formatting differences rather than entirely incorrect entities. LayoutLMv3 also benefits slightly on item names. For prices and totals, normalization has little effect because numeric strings are less sensitive to casing and spacing.

The precision-recall pattern is also informative. LayoutLMv3’s menu-name recall is high because most item-name tokens are visible and tagged when gold OCR is available, but precision is lower because the model sometimes includes extra nearby tokens or duplicate fragments. Donut’s menu-name precision and recall are both lower, which suggests a harder failure mode: it may omit an item entirely or generate a name that is plausi-

The result also suggests that future work should report OCR source explicitly, because “OCR-dependent model performance” can mean very different things depending on whether the OCR is human-verified, commercial, or open-source.

The magnitude of the Tesseract drop is larger than we expected at the proposal stage. It indicates that receipt parsing is not only a semantic labeling task; it is also a text acquisition task. Tesseract errors include missed low-contrast words, merged columns, incorrect reading order, and substitutions in short numeric strings. These errors are especially damaging for receipts because the downstream labels are attached to short spans. If the OCR misses a one-token price, there is no surrounding phrase from which the classifier can recover it. If OCR reads a key such as “TOTAL” incorrectly, the aggregate field may become invisible to a token classifier even though the image still contains it.

This result does not mean OCR-dependent models are inherently unsuitable. It means their deployment story depends on the OCR component. A production system could use a stronger commercial OCR model, train an OCR model on receipts, or jointly fine-tune the extraction model with noisy OCR. The key point is that the post-OCR benchmark setting should be treated as an upper bound unless the OCR source is available at deployment time. For bill splitting, where users may photograph crumpled receipts under restaurant lighting, this distinction is central rather than incidental.

4.3 Hybrid and Label-Construction Ablations

Table 3 evaluates whether CHEF’s improvement comes from meaningful field routing. The reversed hybrid routes menu fields to Donut and aggregate fields to LayoutLMv3. It reaches only 0.874 normalized micro-F1, below Donut alone and well below CHEF. This negative control supports the claim that the routing direction matters: menu fields should be handled by the token classifier when good OCR is available, and aggregate fields should be handled by Donut.

Table 4 reports the `is_key` label-construction ablation for LayoutLMv3. Without filtering key tokens to background, the model performs reasonably on menu fields but fails on aggregate totals because it predicts key-value phrases instead of values. Filtering keys increases normalized micro-

Table 3: Routing-direction ablation. The reversed hybrid performs worse, showing that the gain is not caused by combining models arbitrarily.

System	Normalized Micro-F1
LayoutLMv3 alone	0.937
Donut alone	0.878
CHEF (ours)	0.940
CHEF reversed	0.874

Table 4: Effect of filtering CORD key tokens to background for value extraction.

Variant	Micro-F1	total F1
No <code>is_key</code> filter	0.787	0.010
With <code>is_key</code> filter	0.937	0.898

F1 from 0.787 to 0.937 and increases total-price F1 from 0.010 to 0.898. This was one of the most important engineering findings of the project. The architecture did not change; the improvement came from aligning the labels with the value-extraction objective.

This ablation is a useful reminder that document understanding performance is sensitive to annotation semantics. In CORD, keys and values are both meaningful receipt text, but the target task in our report is value extraction. A token classifier trained to mark both key and value tokens as the same class learns a task that is subtly different from the one we evaluate. The resulting model may appear to understand the receipt while still producing strings that cannot be used as numeric totals. Filtering key tokens resolves this mismatch and makes the model’s training labels match the final API expected by the bill-splitting pipeline.

4.4 Checksum and Qualitative Analysis

Checksum consistency provides a different view of the models. LayoutLMv3, Donut, and CHEF all have consistency rates near the apparent dataset ceiling because many real receipts do not satisfy item-sum equals total due to tax, tips, discounts, or missing non-menu charges. Even CORD’s own ground truth passes the simple checksum only about 53% of the time. Therefore, consistency rate alone is not a reliable model ranking metric.

Mean absolute checksum error is more diagnostic. LayoutLMv3’s mean absolute error is about 5.9% when numeric fields are defined. Donut’s

mean absolute error is far larger because it occasionally hallucinates totals or prices that are orders of magnitude wrong. This behavior is mostly invisible in field-level F1: a model can have high total F1 on most examples while producing rare catastrophic numeric errors. For a financial application, these rare errors matter. A practical system should use checksum disagreement and cross-model disagreement as warnings that require user confirmation.

We also tested the models qualitatively on real receipt photos outside CORD. On a Kroger receipt, Tesseract misread “TOTAL” as a different word, making the LayoutLMv3 pipeline unusable because its input tokens were already wrong. Donut still produced plausible grocery item strings and prices, though not perfectly. On an India Bazaar receipt, the hybrid system extracted five prices summing to \$34.71, matching its predicted subtotal exactly; in that case both models also agreed on subtotal and total. These examples are anecdotal, but they reinforce the quantitative story: OCR-dependent systems can fail abruptly when OCR fails, while agreement between independent architectures can provide a useful verification signal.

Across the quantitative and qualitative evaluations, we observed several recurring error types. The first is token acquisition error, where OCR misses or corrupts the text before LayoutLMv3 sees it. The second is association error, where an item name and price are both detected but paired incorrectly. This is particularly likely when receipts use multi-line item names, modifiers, or columns with weak horizontal alignment. The third is schema error, where a model confuses subtotal, tax, discount, and total lines. The fourth is generation error, which is most visible in Donut: the output follows the expected JSON-like structure but contains a value that is not present in the image.

These error types have different implications for user interaction. OCR acquisition errors may require the user to retake a photo or manually type a missing item. Association errors are easier to correct if the interface shows the receipt image next to the parsed item list. Schema errors can often be detected by comparing subtotal, tax, and total fields. Generation errors are the most concerning because they can look confident and well formatted. This is why we view checksum and cross-model

agreement as deployment aids rather than merely research metrics.

4.5 Open Questions and Next Steps

The experiments do not fully answer how the systems would perform in a deployed mobile application. The CORD test set contains clean benchmark images and annotations, while user photos introduce blur, perspective distortion, shadows, folds, and background clutter. Our small real-world tests suggest the same failure modes but are not large enough to estimate production accuracy. A natural next step is to collect a small evaluation set of real phone-captured receipts with item-level annotations and compare gold OCR, Tesseract, commercial OCR, Donut, LayoutLMv3, and CHEF under the same protocol.

The experiments also leave open the best way to train a hybrid model. CHEF is deliberately simple: it routes fields using fixed rules and does not learn when to trust a model. A stronger system could estimate confidence from token probabilities, decoder likelihood, checksum agreement, and cross-model agreement. It could then decide per receipt or per field whether to accept Donut, accept LayoutLMv3, ask the user to verify, or run a fallback OCR engine. Another direction is to fine-tune Donut specifically for item-level extraction rather than using a public CORD-finetuned checkpoint, and to fine-tune LayoutLMv3 with OCR noise augmentation so it is less brittle to realistic OCR errors.

Finally, the metrics could be expanded. Exact and normalized F1 are useful, but they do not fully capture semantic equivalence in item names, numeric tolerance in prices, or user-correctability. For bill splitting, a system that extracts “latte” instead of “iced latte” may still be usable, while a one-digit price error is not. Future evaluation should weight errors by downstream financial impact and measure how much user correction time each model actually saves.

Another limitation is that CHEF’s field routing was chosen after inspecting model behavior on this task. The reversed-routing ablation shows that the choice is meaningful, but it does not prove that the same routing would be optimal for every dataset or receipt domain. A dataset with poor item-level OCR but very regular menus might favor Donut for more fields. A dataset with excellent

OCR and many visually similar totals might favor LayoutLMv3 more broadly. The general research direction is therefore not the fixed rule itself, but the idea that document extraction systems should route fields according to the type of evidence each field requires.

The project also does not fully explore training cost and latency. Donut avoids OCR, but autoregressive decoding can be slower than token classification. LayoutLMv3 requires an OCR pass before inference, but once tokens are available, token classification is straightforward. A bill-splitting app needs interactive latency on a phone or server-backed workflow, so future work should measure runtime, memory, and energy usage alongside accuracy. A slightly less accurate model might be preferable if it runs locally and preserves privacy, while a more accurate cloud OCR pipeline may raise cost and data-handling concerns.

5 Conclusions

The main takeaway is that receipt parsing is a composite task, and the best architecture depends on the field being extracted and the quality of the OCR source. With verified OCR, LayoutLMv3 is the strongest model for dense item-level fields and reaches 0.886 line-item matching accuracy on CORD. Donut is weaker on line items but slightly stronger on subtotal and total fields. CHEF exploits this complementarity with simple field routing and achieves the best overall normalized micro-F1 of 0.940. However, the OCR-source ablation is the most practically important result: replacing gold OCR with Tesseract drops LayoutLMv3 by roughly 0.62 micro-F1, showing that standard post-OCR benchmark results can substantially overstate deployable performance.

There are several ethical and social implications for automated receipt parsing. The most direct issue is financial accountability: extraction errors can cause people to be charged unfairly, so a bill-splitting application should expose the parsed receipt, highlight uncertain fields, and make correction easy rather than silently assigning costs. Privacy is also important because receipts reveal locations, purchases, habits, and sometimes partial payment information; images and parsed outputs should be stored minimally and protected carefully. Fairness concerns arise if the system works better on receipts from some countries, languages,

stores, or printing styles than others. Finally, transparency matters: users should know when an OCR or neural model has inferred a value rather than directly reading it, especially when checksum or cross-model agreement indicates a possible error.

6 Bibliography

[1] Kim, G., Hong, T., Yim, M., Nam, J., Park, J., Yim, J., Hwang, W., Yun, S., Han, D., Park, S.: OCR-Free Document Understanding Transformer. In: European Conference on Computer Vision (ECCV), pp. 498–517. Springer (2022)

[2] Huang, Y., Lv, T., Cui, L., Lu, Y., Wei, F.: LayoutLMv3: Pre-training for Document AI with Unified Text and Image Masking. In: Proc. 30th ACM Int. Conf. on Multimedia, pp. 4083–4091. ACM (2022)

[3] Xu, Y., Li, M., Cui, L., Huang, S., Wei, F., Zhou, M.: LayoutLM: Pre-training of Text and Layout for Document Image Understanding. In: Proc. 26th ACM SIGKDD Conf., pp. 1192–1200. ACM (2020)

[4] Xu, Y., Xu, Y., Lv, T., Cui, L., Wei, F., Wang, G., Lu, Y., Florencio, D., Zhang, C., Che, W., et al.: LayoutLMv2: Multi-modal Pre-training for Visually-rich Document Understanding. In: Proc. 59th Annual Meeting of the ACL, pp. 2579–2591 (2021)

[5] Park, S., Shin, S., Lee, B., Lee, J., Surh, J., Seo, M., Lee, H.: CORD: A Consolidated Receipt Dataset for Post-OCR Parsing. In: Document Intelligence Workshop at NeurIPS (2019)

[6] Huang, Z., Chen, K., He, J., Bai, X., Karatzas, D., Lu, S., Jawahar, C.V.: ICDAR2019 Competition on Scanned Receipt OCR and Information Extraction. In: Int. Conf. on Document Analysis and Recognition (ICDAR), pp. 1516–1520. IEEE (2019)

[7] Sun, H., Kuang, Z., Yue, X., Lin, C., Zhang, W.: Spatial Dual-Modality Graph Reasoning for Key Information Extraction. arXiv preprint arXiv:2103.14470 (2021)

[8] Liu, Z., Lin, Y., Cao, Y., Hu, H., Wei, Y., Zhang, Z., Lin, S., Guo, B.: Swin Transformer: Hierarchical Vision Transformer Using Shifted Windows. In: Proc. IEEE/CVF Int. Conf. on Computer Vision (ICCV), pp. 10012–10022 (2021)

[9] Lee, K., Joshi, M., Turc, I., Hu, H., Liu, F., Eisen-schlos, J., Khandelwal, U., Shaw, P., Chang, M.W., Toutanova, K.: Pix2Struct: Screenshot Parsing as Pretraining for Visual Language Understanding. In: Int. Conf. on Machine Learning (ICML), pp. 18893–18912 (2023)

[10] Hu, C.: OCRcpt: Receipt Reader and Bill Splitting iOS App Using OCR via Google Vision API. <https://github.com/coreyhu/OCRcpt> (2020)

739 [11] Won, B.: Receipt Hacker: Mobile App
740 for Instant Receipt-Based Cost Splitting.
741 [https://github.com/brendawon/](https://github.com/brendawon/receipt-hacker)
742 [receipt-hacker](https://github.com/brendawon/receipt-hacker) (2021)